

PORTAFOLIO TÉCNICO: MÓDULO 3

El Mundo Real: Problemas Físicos No Lineales

Resolviendo la Ecuación del Calor con Propiedades Variables

Autor: Felipe Wasaff

Fundador, Wasaff Consulting

Índice

Parte A: Formulación Teórica del Problema	2
1. Introducción: Abrazando la Complejidad del Mundo Real	2
2. La Naturaleza de la No Linealidad en la Ecuación del Calor	2
3. Por Qué Fracasa el Enfoque Lineal	3
4. El Enfoque Iterativo: El Método de Newton-Raphson	4
4.1. Aplicando Newton a Nuestra Forma Débil	4
Parte B: Plan de Implementación	5
5. Implementación de un Problema No Lineal en FEniCS	5
5.1. Caso de Estudio: Tratamiento Térmico de un Disco de Acero	5
5.2. Plan de Implementación en FEniCS	5

Parte A: Formulación Teórica del Problema

1. Introducción: Abrazando la Complejidad del Mundo Real

En los Módulos 1 y 2, construimos una base sólida para resolver problemas de conducción de calor en estado estacionario y transitorio. Sin embargo, lo hicimos bajo una suposición simplificadora crucial: que las propiedades del material (k, ρ, c_p) eran constantes. En el mundo real, esta rara vez es la situación.

Este módulo aborda el siguiente nivel de realismo físico: los ****problemas no lineales****. Un problema se vuelve no lineal cuando las propiedades que gobiernan su comportamiento dependen de la propia solución que buscamos. Para la Ecuación del Calor, esto ocurre comúnmente cuando la conductividad térmica depende de la temperatura, $k = k(T)$.

Resolver este tipo de problemas requiere un salto conceptual y matemático. Ya no podemos ensamblar una única matriz y resolver un sistema de ecuaciones lineales. Debemos adoptar un enfoque **iterativo**. Presentaremos el **Método de Newton**, un algoritmo poderoso y universal que forma la columna vertebral de la mayoría de los solvers de elementos finitos para problemas no lineales.

CONTEXTO INGENIERIL

La no linealidad es la norma, no la excepción, en la ingeniería de vanguardia. Consideremos:

- **Procesamiento de Materiales:** En tratamientos térmicos de aceros, la conductividad térmica y la capacidad calorífica varían drásticamente con la temperatura, afectando la velocidad de enfriamiento y las propiedades finales del material.
- **Electrónica de Potencia:** La resistividad del silicio en un semiconductor cambia con la temperatura. A medida que se calienta por el paso de corriente, su capacidad para disipar ese calor también cambia, un acoplamiento no lineal crítico.
- **Ablación Térmica:** En escudos térmicos para reentrada atmosférica, las propiedades del material cambian a temperaturas extremas, un comportamiento altamente no lineal que determina la supervivencia del vehículo.

Dominar los métodos no lineales nos permite simular la física con una fidelidad mucho mayor.

2. La Naturaleza de la No Linealidad en la Ecuación del Calor

Recordemos la forma fuerte de la Ecuación del Calor del Módulo 2:

$$\rho c_p \frac{\partial T}{\partial t} - \nabla \cdot (k \nabla T) = f$$

Hasta ahora, hemos tratado a k como una constante. Sin embargo, para la mayoría de los metales, la conductividad térmica es una función de la temperatura. Una aproximación común es una relación lineal:

$$k(T) = k_0(1 + \alpha(T - T_{ref})) \quad (1)$$

donde k_0 es la conductividad a una temperatura de referencia T_{ref} y α es el coeficiente de temperatura de la conductividad.

Cuando sustituimos $k(T)$ en la Ecuación del Calor, el término de difusión se convierte en:

$$\nabla \cdot (k(T) \nabla T)$$

Este término es ahora ****no lineal**** porque $k(T)$ depende de la solución T . Ya no podemos simplemente “sacar a k fuera de la derivada”. La ecuación ahora relaciona a T consigo misma de una manera más compleja que una simple proporción.

3. Por Qué Fracasa el Enfoque Lineal

Recordemos que al final del proceso de discretización espacial y temporal del Módulo 2, llegábamos a un sistema de ecuaciones algebraicas que, para cada paso de tiempo, podíamos escribir en forma matricial como $\mathbf{A}\mathbf{T}^n = \mathbf{b}$, donde:

- $\mathbf{A}$ era la “matriz del sistema”, que dependía de la malla y las propiedades constantes (k, ρ, c_p) .
- $\mathbf{T}^n$ era el vector de temperaturas nodales desconocidas.
- $\mathbf{b}$ era el “vector de cargas”, que dependía de las fuentes y del estado anterior $\mathbf{T}^{n-1}$.

El punto clave es que podíamos ensamblar $\mathbf{A}$ *una sola vez* al principio de la simulación.

Cuando $k = k(T)$, la matriz del sistema $\mathbf{A}$ ahora depende de la solución $\mathbf{T}^n$ que estamos tratando de encontrar: $\mathbf{A}(\mathbf{T}^n)\mathbf{T}^n = \mathbf{b}$. Esto rompe completamente el paradigma anterior. No podemos construir la matriz sin conocer la solución, y no podemos encontrar la solución sin tener la matriz. Es un problema circular que exige un nuevo enfoque.

4. El Enfoque Iterativo: El Método de Newton-Raphson

El Método de Newton-Raphson (comúnmente, Método de Newton) es una técnica para encontrar las raíces de una función o un sistema de funciones no lineales. Su idea central es brillante por su simplicidad:

1. Partir de una suposición inicial para la solución.
2. Linealizar el problema en torno a esa suposición (aproximar la función no lineal por su tangente).
3. Resolver este problema lineal aproximado para encontrar una mejor corrección.
4. Actualizar la suposición y repetir el proceso hasta que la solución converja.

Para un sistema de ecuaciones no lineales $\mathbf{F}(\mathbf{u}) = \mathbf{0}$, la fórmula de iteración de Newton es:

$$\mathbf{u}_k = \mathbf{u}_{k-1} - \mathbf{J}(\mathbf{u}_{k-1})^{-1} \mathbf{F}(\mathbf{u}_{k-1}) \quad (2)$$

donde $\mathbf{u}_k$ es la solución en la iteración k , $\mathbf{F}(\mathbf{u}_{k-1})$ es el **residuo** (cuán lejos estamos del cero) y $\mathbf{J}$ es la **matriz Jacobiana**, que es la derivada de $\mathbf{F}$ respecto a $\mathbf{u}$.

4.1. Aplicando Newton a Nuestra Forma Débil

Nuestro primer paso es reescribir la forma débil de la ecuación del calor no lineal como un **residuo** que debe ser cero para la solución correcta. Para una iteración k dentro de un paso de tiempo n , y una solución tentativa T_k^n :

$$F(T_k^n; v) = \int_{\Omega} (\rho c_p T_k^n v + \Delta t k(T_k^n) \nabla T_k^n \cdot \nabla v) dV - \int_{\Omega} \rho c_p T^{n-1} v dV - \dots = 0 \quad (3)$$

Para encontrar una mejor solución, $T_{k+1}^n = T_k^n + \delta T$ (donde δT es la corrección), linealizamos este residuo. Esto requiere calcular la **derivada de Gâteaux**, que es el análogo del Jacobiano para formas variacionales. Al hacerlo (ver [?] para detalles), obtenemos un problema lineal para la **corrección** δT :

$$J(\delta T; T_k^n, v) = -F(T_k^n; v) \quad (4)$$

donde J es la forma bilineal Jacobiana.

WASAFFCOPPER

Problema Lineal a Resolver en Cada Iteración de Newton:

Dada una solución tentativa T_k^n , encontrar la corrección δT tal que para todo $v \in \hat{V}$:

$$\underbrace{\int_{\Omega} (\rho c_p \delta T v + \Delta t (k'(T_k^n) \delta T \nabla T_k^n + k(T_k^n) \nabla \delta T) \cdot \nabla v) dV}_{\text{Jacobiano, } J(\delta T; v)} = \underbrace{-F(T_k^n; v)}_{\text{Residuo negativo}} \quad (5)$$

Aunque la Ecuación 5 parece intimidante, FEniCS es capaz de calcular el residuo y el Jacobiano **automáticamente** a partir de la forma del residuo no lineal (Ecuación 3), lo cual es una de sus características más potentes.

Parte B: Plan de Implementación

5. Implementación de un Problema No Lineal en FEniCS

En esta parte, delineamos el plan para implementar un caso de estudio realista que encapsula los conceptos de no linealidad.

5.1. Caso de Estudio: Tratamiento Térmico de un Disco de Acero

5.2. Plan de Implementación en FEniCS

La estructura del código será similar a la del Módulo 2, pero con una diferencia fundamental en el paso de resolución. En lugar de una simple llamada a 'solve()', el solver no lineal de FEniCS ejecutará internamente el bucle del método de Newton.

Estructura del Script de Simulación No Lineal

```

1 # 1. Importar librerías.
2 # 2. Definir parámetros físicos, incluyendo la expresión para k(T).
3 # 3. Generar la malla (usando mshr para el disco) y definir el Espacio de
   Funciones.
4 # 4. Definir las Condiciones de Contorno de Dirichlet.
5 # 5. Definir las funciones para la solución. NOTA: T_n (anterior) y T (
   actual)
6 #     serán ahora Function (no TrialFunction), porque T participa en k(T).
7 # 6. Definir la forma débil del RESIDUO, F, como en la Ecuación 10.
8 # 7. Preparar bucle temporal y archivos de salida.
9 # 8. Bucle temporal:
10 #     - solve(F == 0, T, bcs, solver_parameters={"newton_solver":...})
11 #     - Guardar y actualizar T_n.assign(T)

```

En nuestra próxima sesión de trabajo, desarrollaremos el código completo y analizaremos los fascinantes resultados de esta simulación no lineal.